

MAETTrack: Unleashing the Potential of Pretrained Geometric Priors for 3D Single Object Tracking

Sifan Zhou^{†a,b,*}, Qiwei Wang^{†c}, Linyue Tan^{†d}, Ziyu Liu^d, Ziyu Zhao^{a,b}, Xiaobo Lu^{a,b,*}

^a*School of Automation, Southeast University, Nanjing, 210096, China*

^b*Key Laboratory of Measurement and Control of Complex Systems of Engineering,
Ministry of Education, Nanjing, 210096, China*

^c*Harbin Institute of Technology Shenzhen, Shenzhen, 518000, China*

^d*University of Pennsylvania, Philadelphia, PA, 19104, USA*

Abstract

Large-scale pre-training has transformed representation learning in 2D vision, yet its transferability to 3D single object tracking (SOT) remains insufficiently understood. Directly fine-tuning self-supervised 3D encoders, such as masked autoencoders (MAE), often leads to sub-optimal adaptation because the reconstruction objective is not fully aligned with the spatial-temporal matching requirements of tracking. In this paper, we observe that this difficulty can be interpreted as a layer-wise transfer mismatch: shallow layers tend to preserve transferable geometric cues, while deeper layers become increasingly specialized to the reconstruction pretext task and are less suitable for downstream tracking. Based on this observation, we propose **MAETTrack**, a lightweight adaptation framework for transferring pre-training MAE representations to 3D SOT. **MAETTrack** includes **Layer-Selective Initialization (LSI)**, which initializes only the shallow stages of the tracking backbone from pre-trained weights while re-initializing deeper stages, and **Geometric Residual Gating (GRG)**, which reinforces structurally salient regions in the search BEV features before template-search fusion through residual spatial modulation. Extensive experiments on standard 3D SOT benchmarks show that MAETTrack consistently improves upon vanilla fine-tuning baselines with limited computational overhead. More broadly, our results suggest that effective transfer from 3D reconstruction pre-training to 3D tracking is not merely a matter of partial fine-

*Corresponding author: Sifan Zhou, sifanjay@gmail.com; Xiaobo Lu, xblu2013@126.com. [†] Equal Contribution.

tuning, but depends on a tracking-oriented transfer principle that preserves shallow geometry while adapting deeper representations to the downstream objective. Code will be released after acceptance.

Keywords: 3D single object tracking, LiDAR point clouds, masked autoencoder, transfer learning, Deep Learning for Visual Tracking

1. Introduction

Large-scale self-supervised pre-training has fundamentally reshaped the landscape of 2D computer vision, empowering downstream applications with robust and generalizable representations in computer vision [1, 2, 3], and robotics [4, 5, 6]. Inspired by this success, masked autoencoding methods have significantly advanced 3D representation learning by reconstructing masked observations from partial point clouds [7, 8, 9]. Through this reconstruction-based pre-training objective, MAE-style models learn rich geometric priors and strong sensitivity to local topology and structural continuity, achieving strong performance in scene understanding tasks such as 3D object detection [10, 11].

However, transferring such pretrained representations to 3D Single Object Tracking (SOT) remains challenging. Unlike static perception tasks, 3D SOT requires fine-grained, instance-level spatial-temporal reasoning to continuously localize a target under sparse, partial, and occluded observations. Given the tracked target in the first frame of a point cloud sequence, the goal of 3D SOT is to continuously localize the same object in subsequent frames [12, 13]. In practice, the quality of target representations can degrade substantially when the observed geometry becomes weak or partially missing, making robust feature transfer especially important for 3D SOT.

We argue that this difficulty stems from a fundamental layer-wise objective mismatch between the reconstruction-based pre-training and the downstream tracking objective. Masked reconstruction favors dense and position-sensitive geometric recovery, which may cause deeper representations to become increasingly specialized to semantic or structural patterns useful for inferring missing observations. While beneficial for holistic perception, such deep representations may be less compatible with the strict spatial and

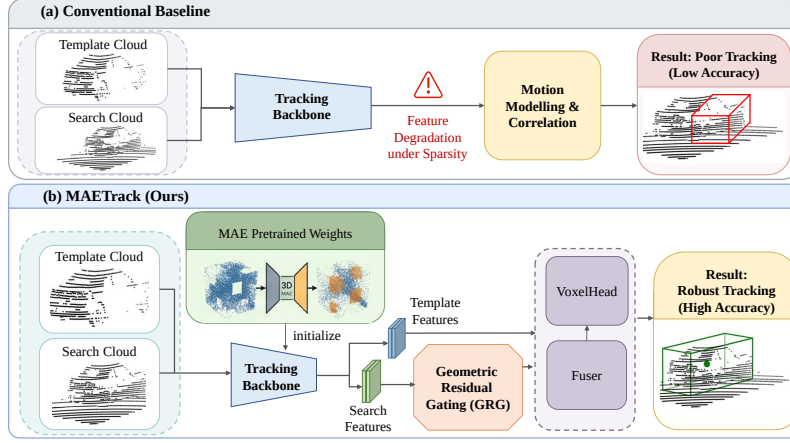


Figure 1: **Comparison between the conventional baseline and our proposed MAETrack.** (a) Conventional trackers suffer from severe feature degradation under point cloud sparsity and ultimately leading to poor tracking accuracy. (b) Our MAETrack overcomes this by initializing the backbone with MAE-pretrained weights for robust feature extraction, and introducing a Geometric Residual Gating (GRG) module to refine the search features, achieving robust tracking with better accuracy and efficiency.

motion consistency required by tracking-oriented matching. Conversely, the shallow layers of pre-trained models tend to preserve more transferable geometric and boundary cues. This suggests a key but underexplored phenomenon: **different layers of MAE-pretrained models encode fundamentally different types of geometric information, with shallow layers being more transferable and deep layers being more task-specific.** From this perspective, 3D SOT relies more heavily on shallow geometric priors for accurate localization under sparse or partially occluded observations, whereas deep reconstruction-specialized semantic representations may introduce misalignment with tracking objectives. This layer-wise discrepancy therefore provides a natural motivation for selective transfer of MAE-pretrained representations.

The prevailing paradigm for using pretrained models in 3D SOT is vanilla fine-tuning, which optimizes the downstream tracking objective in an end-to-end manner. However, this direct transfer strategy lacks an explicit mechanism for preserving and adapting pretrained geometric priors to tracking-oriented features. As a result, the geometric knowledge learned during MAE pre-training can be under-utilized during downstream

optimization. This limitation becomes more evident in challenging tracking scenarios, such as sparse, ambiguous, or partially occluded target observations, where stronger geometric support may be especially beneficial. More importantly, the core bottleneck is not merely optimization instability, but the mismatch between what different pre-trained layers encode and what downstream tracking actually requires. Therefore, an effective solution should account for this transfer gap rather than rely solely on stronger end-to-end fine-tuning.

To address this issue, we propose **MAETrack**, a simple yet effective adaptation framework that bridges reconstruction-oriented pre-training and 3D single object tracking. Recognizing the layer-wise objective mismatch between the two tasks, MAETrack introduces a **Layer-Selective Initialization (LSI)** strategy. Instead of conventional full-network fine-tuning, LSI initializes only the shallow layers of the tracking backbone with BEV-MAE weights to inherit transferable geometric edges and occupancy cues, while randomly initializing the deeper layers to reduce the influence of reconstruction-specific semantics. To further exploit these inherited geometric properties during downstream matching, we design **Geometric Residual Gating (GRG)**, a lightweight spatial modulation module applied to the search BEV features prior to template-search fusion. Formulated as a self-driven residual scaling mechanism, GRG modulates the search representation based solely on its own spatial context, enabling the tracker to reinforce structurally salient regions without disturbing the main spatial-temporal pipeline. We validate MAETrack on standard 3D single object tracking benchmarks, showing consistent improvements over conventional full-network fine-tuning baselines in both Success and Precision. Our study suggests that the key challenge in transferring MAE-pretrained representations to 3D SOT is not whether geometric priors are useful, but how they should be adapted to the downstream tracking objective. Our contributions can be summarized as:

- We identify the layer-wise adaptation gap between reconstruction-based pre-training and downstream tracking, revealing that conventional full-network fine-tuning suffers from potentially mismatched deep representations.
- We propose **MAETrack**, a lightweight 3D SOT framework built upon **Layer-**

Selective Initialization (LSI) and **Geometric Residual Gating (GRG)**. LSI prevents negative semantic transfer by selectively inheriting shallow geometric weights, while GRG achieves a safe, spatially-aware adaptation of these priors via dynamic residual gating mechanism on the search prior to temporal fusion.

- We conduct extensive experiments on standard 3D single object tracking benchmarks and demonstrate that MAETrack consistently improves performance over the strong baseline with negligible overhead.

2. Related Work

2.1. Point Cloud 3D Single Object Tracking.

3D SOT on LiDAR point clouds has evolved from early Siamese paradigms to recent motion-centric methods [14, 15, 16]. Early works framed object tracking as a similarity matching problem within a Siamese architecture. For instance, SC3D [17] utilized a Siamese network to determine the target state through exhaustive feature distance ranking between the template and candidate seeds. To enhance efficiency, 3D-SiamRPN [18] introduced the Region Proposal Network (RPN) into the 3D domain, while P2B [19] leveraged a Hough voting strategy to generate high-quality candidates in an end-to-end manner. Subsequent research has sought to refine this paradigm by incorporating sophisticated structural priors and attention mechanisms; for instance, BAT [20] integrated box-level size priors, while PTT [21] introduced transformer architectures to capture long-range dependencies. To address the inherent sparsity and occlusion challenges, MBPTrack [22] utilized an external memory bank to aggregate multi-frame information. Despite these advancements, appearance matching-based methods remain sensitive to the lack of discriminative textures in LiDAR point clouds. This has catalyzed a shift toward motion-centric paradigms that model inter-frame dynamics. M²Track [23, 12] pioneered this direction by simplifying tracking into 4-DOF relative motion regression, and P2P [24] proposed a part-to-part motion modeling framework that serves as a strong baseline for modern 3D SOT.

However, both Siamese-based matching and motion-based regression currently rely heavily on task-specific supervision from labeled tracking datasets. While self-

supervised learning, particularly Masked Autoencoders (MAE) [25], has shown remarkable potential in learning robust, generalized geometric representations for 3D scene understanding, its application within the 3D SOT pipeline remains largely unexplored. Specifically, the community has yet to explicitly investigate how to transfer MAE-pretrained geometric knowledge to enhance tracking robustness against sparsity and occlusion, leaving the gap in leveraging large-scale self-supervised priors for real-time object tracking.

2.2. 3D Vision MAE.

Self-supervised pre-training via Masked Autoencoders (MAE) has revolutionized visual representation learning. Pioneered by MAE [25] in the 2D domain, this approach demonstrated that reconstructing masked image patches is a powerful pretext task for learning high-level semantic abstractions with remarkable scalability. However, the direct migration of this paradigm to 3D vision is non-trivial due to the inherent sparsity and unstructured nature of point clouds. To bridge this gap, point- and voxel-based variants such as Point-MAE [7] and Voxel-MAE [8] were developed, which adapt the masking strategy to 3D coordinate spaces or discretized grids. Subsequent research has further diversified the MAE landscape: GeoMAE [26] incorporates geometric constraints for finer structural modeling, while BEV-MAE [27] and Multi-view MAE explore masked reconstruction at the bird’s-eye-view or multi-perspective levels for large-scale outdoor scenes. While these methods have proven effective in static 3D perception tasks—such as object classification and detection [10]—their utility is predominantly measured by their ability to capture intra-frame geometric integrity.

However, static success does not seamlessly translate to 3D SOT, which demands discriminative cross-frame association over mere geometry reconstruction. The generative nature of MAE-based priors may conflict with tracking’s discriminative requirements, particularly as focus shifts to motion modeling in deeper layers. Consequently, the synergy between self-supervised structural learning and real-time localization remains virtually unexplored in the 3D SOT community.

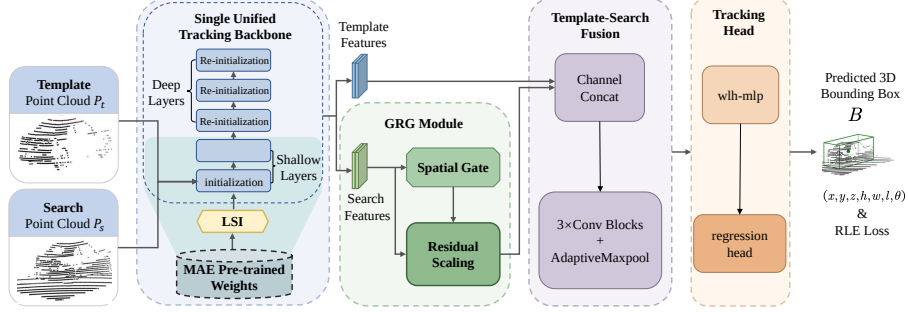


Figure 2: Framework of the proposed MAETrack. The pipeline takes a template point cloud and a search point cloud as inputs with an unified feature extraction. To inherit robust geometric priors, its shallow layers are initialized from MAE-pre-trained weights via Layer-Selective Initialization (LSI), while the deep layers are randomly re-initialized to adapt to the tracking task. Then, the search features are fed into the Geometric Residual Gating (GRG) module, which learns a spatial gating map through a spatial gate to selectively enhance informative geometric patterns via residual scaling. Finally, the template and refined search features are aggregated in the template-search fusion module and input the tracking head to predict the 3D bounding box.

3. Methodology

3.1. Problem Formulation

Given a sequence of 3D point clouds, the goal of 3D single object tracking (SOT) is to continuously localize a specific target, typically initialized by a 3D bounding box $\mathbf{B} = \{\mathbf{b} = [x, y, z, h, w, l, \theta]^T \in \mathbb{R}^{1 \times 7}\}$ in the first frame. Many recent frameworks [28, 29] rely on template-search feature extraction and fusion to predict the target state frame by frame. Let $\mathcal{P}_{temp}$ and $\mathcal{P}_{search}$ denote the point clouds of the template (target) and the search region, respectively, where $\mathcal{P}_{temp} = \{\mathbf{p}_i^t = [x_i, y_i, z_i, r_i]^T \in \mathbb{R}^{N \times 4}\}$, where x_i, y_i, z_i denote the coordinate values of each point. A shared backbone extracts their Bird’s-Eye-View (BEV) features, which are subsequently fused for target localization. In our setting, the prediction head mainly regresses the target center and heading angle, while the box size is initialized from the template commonly used in prior 3D SOT pipelines. Formally, the tracking objective can be formulated as:

$$\mathbf{B}_t = \mathcal{F}_{track}(\mathcal{P}_{temp}, \mathcal{P}_{search}, \mathbf{B}_{t-1}; \Theta), \quad (1)$$

where $\mathcal{F}_{track}$ denotes the learnable tracking network parameterized by Θ , and $\mathbf{B}_t$ represents the predicted target state at frame t .

3.2. Motivation

While pre-trained foundation models [26] provide strong geometric representations, naively fine-tuning the entire network for 3D SOT often leads to negative transfer. This suggests reconstruction-oriented pre-training and downstream tracking optimize different objectives: the former emphasizes recovering missing geometric content, while the latter requires robust target-specific spatial-temporal matching.

From a representation learning perspective, this discrepancy can be attributed to a layer-wise specialization phenomenon, where shallow layers tend to preserve local geometric structures such as occupancy patterns and boundaries, while deeper layers encode reconstruction-oriented semantic abstractions [10, 30]. This leads to a mismatch when directly transferring all layers to tracking tasks, which rely more on instance-level spatial consistency rather than reconstruction fidelity. Based on this observation, we formulate **MAETrack** as a structured representation adaptation problem, where the goal is not to uniformly fine-tune pretrained features, but to explicitly account for the hierarchical structure of MAE representations. In particular, we aim to selectively preserve transferable geometric components in shallow layers while allowing higher-level representations to adapt to tracking-specific objectives.

To formulate this transfer problem, we denote a pretrained MAE backbone with L hierarchical stages as:

$$\Theta^{\text{MAE}} = \{\theta_1^{\text{MAE}}, \theta_2^{\text{MAE}}, \dots, \theta_L^{\text{MAE}}\}, \quad (2)$$

where θ_i^{MAE} represents the parameters of the i -th stage. Conventional fine-tuning directly initializes all layers from the pretrained model:

$$\Theta^{\text{init}} = \Theta^{\text{MAE}}. \quad (3)$$

However, this uniform transfer strategy implicitly assumes that all layers share identical transferability, which contradicts the layer-wise specialization property of MAE representations. To characterize the transfer compatibility of each layer, we define:

$$T_i = \mathcal{D}(\theta_i^{\text{MAE}}, \theta_i^{\text{trk}}), \quad (4)$$

where $\mathcal{D}(\cdot, \cdot)$ measures the representation compatibility between the pretrained and tracking-oriented parameters at the i -th stage. Due to the different optimization objectives, shallow layers generally exhibit higher transfer compatibility than deeper layers: $T_1, T_2, \dots, T_k > T_{k+1}, \dots, T_L$.

Based on this observation, we formulate **MAETrack** as a structured representation adaptation problem, where the goal is not to uniformly fine-tune pretrained features, but to explicitly account for the hierarchical structure of MAE representations. Specifically, we aim to preserve transferable geometric components in shallow layers while allowing higher-level representations to adapt to tracking-specific objectives.

Accordingly, we introduce a layer-wise initialization operator $\mathcal{A}(\cdot)$ to guide the transfer process:

$$\theta_i^{\text{init}} = m_i \theta_i^{\text{MAE}} + (1 - m_i) \theta_i^{\text{rand}}, \quad (5)$$

where m_i is a binary transfer mask. Specifically, $m_i = 1$ indicates that the corresponding layer inherits MAE-pretrained weights, while $m_i = 0$ denotes random initialization. This formulation provides the foundation of our **Layer-Selective Initialization (LSI)**, which preserves shallow geometric priors while decoupling deeper reconstruction-specific representations.

After initialization, all parameters remain trainable during downstream optimization:

$$\Theta^{\text{trk}} = \Theta^{\text{init}} - \eta \nabla_{\Theta} \mathcal{L}_{\text{track}}, \quad (6)$$

where η denotes the optimization learning rate and $\mathcal{L}_{\text{track}}$ represents the tracking objective. Therefore, LSI does not constrain the final parameter space, but instead provides a geometry-aware initialization trajectory for adapting pretrained representations.

We further decompose this layer-wise representation mismatch into two coupled factors that jointly determine the effectiveness of MAE transfer for 3D SOT. *First*, shallow layers encode transferable geometric structures that should be preserved during adaptation, while deep layers encode reconstruction-specific semantics that may hinder tracking performance. This motivates a selective transfer strategy that preserves shallow geometric representations while decoupling deep semantic components, leading to **Layer-Selective Initialization (LSI)**. *Second*, even within transferable shallow representations, not all spatial regions contribute equally to tracking. In particular,

structurally salient regions such as boundaries and occupied areas are more informative under sparse observations. This motivates a feature-level reinforcement strategy, leading to **Geometric Residual Gating (GRG)**, which adaptively enhances informative regions while preserving pretrained geometric consistency. Therefore, we propose **MAETrack**, a lightweight adaptation framework that transfers shallow geometric knowledge while enabling deeper layers to adapt more freely to the downstream tracking objective.

3.3. Methodology: MAETrack

To instantiate the proposed structured representation adaptation principle, MAETrack rethinks the conventional fine-tuning paradigm by shifting from full-network parameter inheritance to a Layer-Selective Initialization (LSI) strategy, followed by feature-level modulation. As illustrated in Figure 2, MAETrack operates on a single unified tracking backbone rather than a dual-stream architecture, ensuring that representation adaptation is performed within a consistent geometric space. Instead of treating all layers uniformly, the framework explicitly aligns with the layer-wise specialization property of MAE representations by partitioning transferable and task-specific components along network depth. Following this principle, MAETrack is composed of two complementary instantiations: **LSI**, which preserves transferable shallow geometric priors while decoupling deep reconstruction semantics, and **Geometric Residual Gating (GRG)**, which performs spatially-aware reinforcement of structurally salient regions in the search BEV features prior to template-search fusion.

3.4. Layer-Selective Initialization (LSI)

The core idea behind LSI is motivated by the hierarchical structure of MAE-pretrained representations. Rather than treating all layers as equally transferable, we observe that different network depths encode fundamentally different types of geometric information due to the reconstruction-oriented pre-training objective. From a representation learning perspective, shallow layers tend to encode generalizable low-level geometric primitives, such as occupancy patterns, local surfaces, and boundary structures, which are relatively invariant across tasks and thus more transferable to

downstream 3D SOT. In contrast, deeper layers are progressively shaped by the reconstruction objective, learning higher-level semantic abstractions that are optimized for recovering missing content rather than precise instance-level spatial matching.

This layer-wise specialization introduces a structural mismatch when directly transferring all pretrained layers to tracking, since 3D SOT requires accurate geometric correspondence and spatial consistency rather than reconstruction fidelity. Therefore, full-network fine-tuning may lead to interference between transferable geometric representations and reconstruction-specific semantics. To address this, we propose **Layer-Selective Initialization (LSI)**, which performs *structure-aware transfer along the network depth*. Formally, consider a sparse 3D convolutional backbone partitioned into L stages, denoted by its parameter sequence $\Theta = \{\theta_1, \theta_2, \dots, \theta_L\}$.

Instead of the conventional full-sequence loading strategy, LSI implements a selective inheritance mechanism along the network depth. Specifically, we introduce a binary transfer mask m_i to determine whether the i -th backbone stage inherits MAE-pretrained parameters:

$$m_i = \begin{cases} 1, & i \leq k, \\ 0, & i > k, \end{cases} \quad (7)$$

where k denotes the number of inherited shallow stages. The initialized parameters of each stage are then formulated as:

$$\theta_i^{init} = m_i \theta_i^{MAE} + (1 - m_i) \theta_i^{rand}, \quad (8)$$

where θ_i^{MAE} represents the parameters from the MAE-pretrained backbone and θ_i^{rand} denotes random initialization. This formulation explicitly models LSI as a layer-wise parameter transfer operator, where transferable geometric representations are inherited from pre-training while reconstruction-specific deep representations are released for task-specific adaptation.

Shallow Geometric Transfer: We initialize shallow layers $\Theta_{\text{shallow}} = \{\theta_1, \dots, \theta_k\}$, where $k < L$, using pretrained weights [27] to preserve transferable geometric structure. These layers define a low-level geometric subspace that is highly relevant for tracking

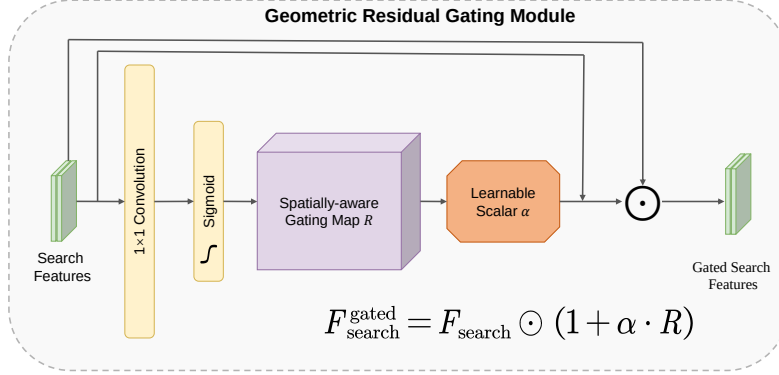


Figure 3: Illustration of Geometric Residual Gating (GRG).

under sparse observations. In our implementation, the shallow initialization covers the first three backbone stages, and the effect of this design is further validated through stage-wise ablations in the experiments.

Deep Representation Re-initialization: The deeper layers $\Theta_{\text{deep}} = \{\theta_{k+1}, \dots, \theta_L\}$ are randomly initialized to decouple them from reconstruction-specific biases, so that network rebuild task-specific high-level representations without being constrained by reconstruction-aware semantics. Crucially, after this hybrid initialization, all parameters are jointly optimized by minimizing the tracking objective:

$$\Theta^* = \arg \min_{\Theta} \mathcal{L}_{\text{track}}(\Theta; \mathbf{P}_{\text{temp}}, \mathbf{P}_{\text{search}}), \quad (9)$$

where Θ includes both inherited shallow layers and randomly initialized deep layers. $\Theta = \Theta_{\text{shallow}} \cup \Theta_{\text{deep}}$ remains fully trainable ($\nabla_{\Theta} \neq 0$) during downstream tracking. Importantly, this design does not freeze any parameters. Instead, after initialization, the entire network remains fully trainable, while the initialization itself constrains the optimization trajectory toward a geometry-preserving subspace. This allows MAETrack to retain transferable shallow geometric priors while avoiding negative transfer from reconstruction-specialized deep representations.

3.5. Geometric Residual Gating (GRG)

While Layer-Selective Initialization (LSI) operates at the backbone level to preserve transferable shallow geometric representations, the intermediate feature representations

in the downstream tracking pipeline may still suffer from partial degradation due to occlusion, sparsity, and background clutter in the search region. This motivates the need for an explicit feature-level mechanism that reinforces structurally informative regions while preserving the pretrained geometric prior. To address this, we propose **Geometric Residual Gating (GRG)**, a lightweight geometry-preserving residual modulation module designed specifically for MAE-pretrained feature adaptation in 3D SOT. Unlike conventional spatial attention mechanisms (e.g., SE [31] or CBAM [32]), which perform generic feature reweighting, GRG is designed to selectively enhance structurally salient regions while maintaining the integrity of pretrained geometric representations.

We apply GRG only to the search branch, as it contains spatially ambiguous and noise-corrupted observations due to background clutter and partial occlusion, whereas the template branch provides a relatively stable geometric reference derived from the first-frame annotation. Therefore, feature reinforcement is more critical in the search region, where MAE-pretrained representations are more likely to be degraded by sparse observations.

The key motivation of GRG is that under MAE-pretrained representations and sparse observations, spatial reliability is highly non-uniform. In particular, structurally salient regions such as object boundaries and occupied areas tend to preserve more stable geometric cues, making them more informative for downstream tracking.

Given a pair of point clouds, let $F_{\text{search}} \in \mathbb{R}^{C \times H \times W}$ and $F_{\text{temp}} \in \mathbb{R}^{C \times H \times W}$ denote the BEV features extracted by the backbone. GRG first estimates a spatial geometric reliability map from the search feature:

$$R_s = \sigma(W_g * F_{\text{search}} + b_g), \quad (7)$$

where W_g and b_g denote the parameters of a lightweight 1×1 convolutional layer and $\sigma(\cdot)$ denotes the Sigmoid activation. Importantly, this gating mechanism does not aim to perform global feature reweighting; instead, it identifies spatial regions where geometric reinforcement is beneficial under sparse and occluded observations. The resulting gate highlights regions where geometric reinforcement is most beneficial for downstream localization. The predicted map $R_s \in \mathbb{R}^{1 \times H \times W}$ represents the spatial reliability of geometric structures in the search region.

Instead of directly replacing the original feature representation, GRG introduces a residual geometric enhancement:

$$\Delta F_{\text{search}} = \alpha(F_{\text{search}} \odot R_s), \quad (8)$$

where α is a learnable scalar controlling the reinforcement strength. This residual formulation ensures that the original MAE-pretrained geometric representation remains the dominant component while allowing adaptive enhancement of informative spatial regions.

The final search feature is obtained as:

$$\tilde{F}_{\text{search}} = F_{\text{search}} + \Delta F_{\text{search}} = F_{\text{search}} \odot (1 + \alpha R_s). \quad (9)$$

The adapted search representation is then fused with the template feature:

$$F_{\text{fusion}} = \mathcal{F}_{\text{conv_head}}(\text{Concat}(F_{\text{template}}, \tilde{F}_{\text{search}})). \quad (10)$$

From a representation learning perspective, GRG can be interpreted as a geometry-preserving adaptation operator, which enhances task-relevant spatial structures while maintaining consistency with pretrained geometric priors. This is particularly important in 3D SOT, where sparse observations and occlusions can easily weaken local geometric signals. Through the synergy with LSI, GRG enables MAETrack to achieve a unified representation-preserving adaptation framework, where LSI operates at the depth level of the backbone, and GRG operates at the spatial level of feature refinement, jointly ensuring effective transfer of MAE-pretrained geometric knowledge to downstream tracking tasks.

3.6. Prediction Head and Training Loss.

After template-search fusion, the fused representation is fed into the tracking head to predict the target state. Following the common 3D SOT setting, we directly regress the target center and orientation using the final fused features, while keeping the baseline’s part-to-part motion modeling and prediction head strictly unchanged. In other words, our modifications are limited to the backbone initialization strategy and the pre-fusion

search-side feature modulation, without altering the downstream motion modeling or supervision design. The entire MAETrack framework is trained end-to-end with the same tracking objective as the baseline. Specifically, we adopt the standard regression losses used in prior 3D SOT works [33]:

$$\mathbf{L}_{\text{track}} = \lambda_1 \mathcal{L}_{(x,y)} + \lambda_2 \mathcal{L}_z + \lambda_3 \mathcal{L}_{\text{rot}}, \quad (11)$$

where λ_1 , λ_2 , and λ_3 balance the position and orientation terms. This design keeps the supervision protocol unchanged and isolates the effect of our transfer strategy to the backbone initialization and search-side feature modulation.

4. Experiments

Dataset and Metrics. We follow the common setup [24, 34] and conduct experiments on the KITTI [35] and large-scale nuScenes [36] dataset. Notably, due to the limited data size of the KITTI (only 19 training, 2 validation sequences) makes it challenging to adequately evaluate the methods. In contrast, the nuScenes dataset comprises 700 training and 150 validation sequences, across 40K point cloud frames, allowing for a more comprehensive evaluation. The evaluation metrics is followed the common setup [21, 28] to report *Success* and *Precision* based on one pass evaluation (OPE) [37, 38].

Implementation Details. Following prior 3D SOT methods [19, 28], we use frame $t - 1$ as the template and frame t as the search frame. The category-dependent crop ranges are $[(-4.8, 4.8), (-4.8, 4.8), (-1.5, 1.5)]$ for cars, $[(-1.92, 1.92), (-1.92, 1.92), (-1.5, 1.5)]$ for pedestrians, and $[(-9.6, 9.6), (-9.6, 9.6), (-3.0, 3.0)]$ for trucks, trailers, and buses. Their voxel sizes are $(0.075, 0.075, 0.15)$, $(0.03, 0.03, 0.15)$, and $(0.15, 0.15, 0.30)$, respectively. Each frame pair generates four candidates through reference-box perturbation, and synchronized horizontal flipping is additionally used on nuScenes. For LSI, the input convolution and the first two residual sparse-convolution stages are initialized from the Waymo-pretrained BEV-MAE checkpoint, while the deeper stages are randomly initialized; all parameters remain trainable. The GRG coefficient α is initialized to 10^{-3} . We use AdamW with an initial learning rate of 2×10^{-4} , weight

decay 10^{-5} , cosine annealing to 10^{-6} , mixed-precision training, and gradient clipping with a maximum norm of 35. The global batch size is 256 on one RTX 4090 GPU. The nuScenes car/pedestrian, truck/bus, and trailer models are trained for 20, 50, and 100 epochs, respectively, while the KITTI models are trained for 50 epochs. Validation is performed every epoch, and the checkpoint with the best Precision is selected. In our stage notation, Stage 1 denotes the input sparse convolution block, while Stages 2–4 denote the subsequent residual sparse-convolution stages. Thus, Stage 1+2+3 corresponds to initializing the input convolution and the first two residual stages from the BEV-MAE [27] pre-trained checkpoint.

Inference. During inference stage, MAETrack predicts relative motion $[\Delta x, \Delta y, \Delta z, \Delta r]$ frame-by-frame. The regressed relative motion is then applied to target box $\mathcal{B}_{t-1}$ in the previous frame to locate the target box $\mathcal{B}_t$ in current frame. Alg. 1 presents the whole inference process of MAETrack in one trajectory. Our experiment video is available at Appendix.

4.1. Comparison with SOTA Method

Results on KITTI Dataset. As summarized in Table 1, we evaluate our MAETrack on the KITTI dataset and compare it with state-of-the-art 3D trackers. It is worth noting that KITTI poses a unique challenge due to its extremely limited training data size (only 19 sequences), which typically makes data-hungry models prone to severe overfitting. Despite this, our MAETrack achieves compelling performance. Specifically, on the Car category, MAETrack attains **75.2%/87.5%** in Success and Precision, respectively, outperforming the strong baseline tracker P2P by **+2.0%/+2.1%**. This significant performance gain explicitly demonstrates the strong data efficiency of our framework: the geometric prior knowledge successfully transferred by our MAE pre-training effectively compensates for the lack of large-scale annotated data, providing a structurally robust representation that prevents the model from overfitting to sparse training samples.

Results on nuScenes Dataset. Table 2 summarizes the tracking results of our MAETrack and other state-of-the-art methods on the large-scale nuScenes dataset [36]. Compared to our strong baseline P2P [24], our MAETrack achieves comprehensive improvements across almost all categories. Specifically, on the Car category, our method

Algorithm 1: Tracking process of MAETrack

Input: A point-cloud sequence $\{\mathcal{P}_t\}_{t=1}^T$; an initial target bounding box

$\mathbf{B}_1 = [x_1, y_1, z_1, h, w, l, \theta_1]^T$ in the first frame; MAE Pre-trained weights Θ^{MAE} ; the number of inherited shallow stages k .

Output: A series of predicted bounding boxes $\{\mathbf{B}_t = [x_t, y_t, z_t, h, w, l, \theta_t]^T\}_{t=2}^T$.

Offline model construction with LSI:

Build a tracking backbone $\Phi(\cdot; \Theta)$ with L stages, where $\Theta = \{\theta_1, \theta_2, \dots, \theta_L\}$;

Initialize shallow stages $\{\theta_1, \dots, \theta_k\}$ from Θ^{MAE} and randomly initialize deeper stages $\{\theta_{k+1}, \dots, \theta_L\}$; Keep all parameters trainable and optimize the whole

MAETrack model with the tracking loss $\mathcal{L}_{\text{track}}$;

Online tracking:

for $t = 2$ **to** T **do**

Crop the template point cloud $\mathcal{P}_{\text{temp}}^{t-1}$ from frame $t-1$ according to $\mathbf{B}_{t-1}$ and the search point cloud $\mathcal{P}_{\text{search}}^t$ from frame t around the previous prediction

$\mathbf{B}_{t-1}$. And convert $\mathcal{P}_{\text{temp}}^{t-1}$ and $\mathcal{P}_{\text{search}}^t$ into voxelized BEV inputs;

Extract template and search features using the LSI-initialized backbone:

$$\mathbf{F}_{\text{template}}, \mathbf{F}_{\text{search}} = \Phi(\mathcal{P}_{\text{temp}}^{t-1}), \Phi(\mathcal{P}_{\text{search}}^t).$$

Predict the spatial gating map from the search feature:

$$\mathbf{R} = \sigma(\mathcal{F}_{\text{gate}}(\mathbf{F}_{\text{search}})).$$

Apply Geometric Residual Gating (GRG) to obtain enhanced search feature:

$$\mathbf{F}_{\text{search}}^{\text{gated}} = \mathbf{F}_{\text{search}} \odot (1 + \alpha \mathbf{R}).$$

Fuse the template feature and the gated search feature:

$$\mathbf{F}_{\text{fusion}} = \mathcal{F}_{\text{conv_head}}(\text{Concat}(\mathbf{F}_{\text{template}}, \mathbf{F}_{\text{search}}^{\text{gated}})).$$

Predict the target center and heading angle using the tracking head and keep the target size (h, w, l) from the template and obtain:

$$\mathbf{B}_t = [\hat{x}_t, \hat{y}_t, \hat{z}_t, h, w, l, \hat{\theta}_t]^T.$$

end

Return $\{\mathbf{B}_t\}_{t=2}^T$.

Table 1: Comparisons with state-of-the-art methods on KITTI dataset [35]. *Success / Precision* are used for evaluation. **Bold** and underline denote the best result and the second-best one respectively. † means re-implementation based on official code.

Paradigm	Tracker	Source	Mean (14,068)	Car (6,424)	Pedestrian (6,088)	Van (1,248)	Cyclist (308)
Scratch	SC3D [17]	CVPR'19	31.2 / 48.5	41.3 / 57.9	18.2 / 37.8	40.4 / 47.0	41.5 / 70.4
	P2B [19]	CVPR'20	42.4 / 60.0	56.2 / 72.8	28.7 / 49.6	40.8 / 48.4	32.1 / 44.7
	3D-SiamRPN[18]	IEEE Sensors J	46.6 / 64.9	58.2 / 76.2	35.2 / 56.2	45.7 / 52.9	36.2 / 49.0
	PTT [39]	IROS'21	55.1 / 74.2	67.8 / 81.8	44.9 / 72.0	43.6 / 52.5	37.2 / 47.3
	V2B [33]	NeurIPS'21	58.4 / 75.2	70.5 / 81.3	48.3 / 73.5	50.1 / 58.0	40.8 / 49.7
	PTTR [40]	CVPR'22	57.9 / 78.2	65.2 / 77.4	50.9 / 81.6	52.5 / 61.8	65.1 / 90.5
	M ² Track [23]	CVPR'22	62.9 / 83.4	65.5 / 80.8	61.5 / 88.2	53.8 / 70.7	73.2 / 93.5
	M ² Track++ [12]	TPAMI'23	66.5 / 85.2	71.1 / 82.7	61.8 / 88.7	62.8 / 78.5	75.9 / 94.0
	GLT-T [41]	AAAI'23	60.1 / 79.3	68.2 / 82.1	52.4 / 78.8	52.6 / 62.9	68.9 / 92.1
	MBPTrack [22]	ICCV'23	70.3 / 87.9	73.4 / 84.8	68.6 / 93.9	61.3 / 72.7	<u>76.7</u> / 94.3
	SyncTrack [42]	ICCV'23	64.1 / 81.9	73.3 / 85.0	54.7 / 80.5	60.3 / 70.0	73.1 / 93.8
	PTTR++ [43]	TPAMI'24	63.9 / 82.8	<u>73.4</u> / 84.5	55.2 / 84.7	55.1 / 62.2	71.6 / 92.8
	VoxelTrack [44]	ACM MM'24	70.4 / 88.3	72.5 / 84.7	67.8 / 92.6	69.8 / <u>83.6</u>	75.1 / 94.7
	VPMCAN [13]	PR'25	70.1 / 88.6	74.6 / <u>86.3</u>	65.8 / 96.2	63.1 / 73.5	76.9 / 96.6
	P2P† [24]	IJCV'25	<u>71.1</u> / <u>88.9</u>	73.2 / 85.4	69.0 / <u>93.4</u>	<u>69.8</u> / 83.3	75.1 / 94.0
	MAETrack	Ours	72.0 / 89.9	75.2 / 87.5	<u>68.8</u> / 93.2	70.5 / 84.6	75.5 / <u>94.7</u>

Table 2: Comparison with SOTA methods on the nuScenes dataset. Success and Precision are used for evaluation, **Bold** and underline denote the best result and the second-best one respectively. * 64159 indicates the number of instances of cars.

Method	Mean (117,278)	Car (64,159)*	Pedestrian (33,227)	Truck (13,587)	Trailer (3,352)	Bus (2,953)
SC3D [17]	20.70 / 20.20	22.31 / 21.93	11.29 / 12.65	35.28 / 28.12	35.28 / 28.12	29.35 / 24.08
P2B [19]	36.48 / 45.08	38.81 / 43.81	28.39 / 52.24	48.96 / 40.05	48.96 / 40.05	32.95 / 27.41
PTT [28]	36.33 / 41.72	41.22 / 45.26	19.33 / 32.03	50.23 / 48.56	51.70 / 46.50	39.40 / 36.70
BAT [20]	38.10 / 45.71	40.73 / 43.29	28.83 / 53.32	52.59 / 44.89	52.59 / 44.89	35.44 / 28.01
V2B [33]	- / -	54.40 / 59.70	30.10 / 55.40	53.70 / 54.50	54.90 / 51.44	- / -
M ² -Track [23]	49.23 / 62.73	55.85 / 65.09	32.10 / 60.92	57.36 / 59.54	57.61 / 58.26	51.39 / 51.44
PTTR [40]	44.50 / 52.07	51.89 / 58.61	29.90 / 45.09	45.30 / 44.74	45.87 / 38.36	43.14 / 37.74
GLT-T [41]	44.42 / 54.33	48.52 / 54.29	31.74 / 56.49	52.74 / 51.43	57.60 / 52.01	44.55 / 40.69
PTTR++ [43]	51.86 / 60.63	59.96 / 66.73	32.49 / 50.50	59.85 / 61.20	54.51 / 50.28	53.98 / 51.22
VoxelTrack [44]	59.00 / <u>71.40</u>	63.90 / 71.60	<u>46.80</u> / <u>75.90</u>	<u>64.80</u> / <u>65.90</u>	69.50 / 64.30	<u>60.10</u> / <u>57.70</u>
P2P† [24]	<u>59.22</u> / 71.19	<u>64.61</u> / <u>71.98</u>	45.64 / 74.62	64.42 / 65.37	<u>70.23</u> / <u>66.08</u>	58.54 / 56.13
MAETrack	61.16 / 73.54	66.05 / 73.57	47.21 / 75.96	68.50 / 70.53	74.10 / 71.63	62.11 / 60.86

reaches 66.05%/73.57%, and on the challenging rigid classes like Truck and Trailer, we achieve significant boosts of +4.38%/+5.16% and +3.87%/+5.55%, respectively. Notably, on the Bus category, our method yields a large gain, reaching an improvement of +3.57% / +4.73% over the baseline. Overall, MAETrack surpasses previous methods and demonstrates that our layer-selective initialization and geometric residual gating effectively transfer geometric priors to significantly enhance tracking performance.

Table 3: Comparison of the running speeds on some representative methods.

Method	P2B[19]	PTT[28]	BAT[20]	PTTR [40]	M ² Track[23]	GLT-T [41]
FPS	40.0	40.0	57.0	43.0	51.2	30.0
Method	VoxelTrack[44]	M ² Track++[12]	BAT [20]	V2B [33]	P2P [24]	MAETrack (our)
FPS	36.0	57.0	57.0	37.0	88.9	84.3

Running Speed: Inference speed is also a vital factor for practical applications. We present a comprehensive speed comparison of MAETrack with other methods in Tab. 3. Following common evaluation protocols [19, 28, 24], speed is measured by calculating the average running time of all frames in the Car category. On a single NVIDIA RTX 4090 GPU, MAETrack achieves 84 FPS. Despite the computational overhead incurred by our GRG module, MAETrack yields significant performance improvements, maintaining a better trade-off between accuracy and speed.

Robustness to Point-cloud Sparsity. To directly validate the robustness of MAETrack under sparse or occluded scenes, we conduct a sparsity analysis on the Car category of KITTI and nuScenes. Specifically, we group tracking sequences according to the number of target points in the first frame and evaluate all frames within each group. The number of sequences and frames in each sparsity interval is also reported in Table 4 to reflect the statistical reliability of each group. As shown in Table 4, MAETrack consistently improves over the strong baseline P2P in most sparse intervals on both datasets. On KITTI, MAETrack achieves clear gains in the highly sparse intervals, improving P2P from 64.8/75.0 to 67.1/77.7 in the [0,10) interval and from 70.7/83.6 to 71.7/85.8 in the [10,20) interval. Similar improvements are observed in the [20,30) and [30,40) intervals, where MAETrack obtains 76.4/89.4 and 83.3/94.1, respectively. On nuScenes, where sparse observations are much more common, MAETrack also shows

Table 4: Comparison on varying levels of sparsity on Car category of KITTI [35] and NuScenes [36]. **Bold** denotes the best result.

Dataset	KITTI Car					
Interval	[0, 10)	[10, 20)	[20, 30)	[30, 40)	[40, 50)	[50, +∞)
Sequence Number	46	29	19	5	3	18
Frame Number	2,394	1,590	709	97	80	1,554
M ² Track [23]	53.0 / 67.1	60.2 / 73.9	62.6 / 75.7	77.6 / 92.1	61.6 / 72.1	79.6 / 92.1
P2P [24]	64.8 / 75.0	70.7 / 83.6	73.5 / 87.8	77.0 / 90.4	80.2 / 92.7	81.2 / 92.7
MAETrack	67.1 / 77.7	71.7 / 85.8	76.4 / 89.4	83.3 / 94.1	64.0 / 71.9	81.5 / 93.3

Dataset	NuScenes Car					
Interval	[0, 10)	[10, 20)	[20, 30)	[30, 40)	[40, 50)	[50, +∞)
Sequence Number	2,884	212	101	64	37	362
Frame Number	45,322	4,375	2,190	1,321	814	10,127
M ² Track [23]	52.1 / 60.9	57.1 / 65.9	65.8 / 73.4	68.1 / 76.3	70.1 / 78.6	75.7 / 83.1
P2P [24]	63.3 / 71.4	64.1 / 71.9	64.3 / 70.2	69.6 / 77.4	73.9 / 79.9	72.8 / 79.4
MAETrack	64.3 / 71.7	65.4 / 73.5	65.9 / 72.7	69.7 / 77.5	73.7 / 81.7	72.9 / 80.6

consistent advantages in low-point regimes, improving P2P from 63.3/71.4 to 64.3/71.7 in the [0,10) interval and from 64.1/71.9 to 65.4/73.5 in the [10,20) interval. These results demonstrate that the proposed LSI and GRG modules are particularly beneficial when the available geometric evidence is weak.

It is also worth noting that the performance gap becomes smaller in dense intervals, where baseline trackers already receive sufficient geometric information from the point cloud. This observation is consistent with our motivation: MAE-pretrained shallow geometric priors and search-side residual gating are most useful when the target observation is sparse, partially missing, or structurally ambiguous. The only exception appears in the [40,50) interval on KITTI, where MAETrack underperforms P2P. However, this interval contains only 3 sequences and 80 frames, making the result statistically less stable. Overall, the sparsity analysis provides direct evidence that MAETrack improves robustness under sparse point-cloud observations rather than only improving category-level average performance.

4.2. Ablation Study

To validate the effectiveness of our MAETrack, we conduct comprehensive ablation studies on KITTI and nuScenes dataset of Car and Pedestrian category. We adopt the strong P2P as baseline and evaluate the contributions of our core designs: Layer-Selective Initialization (LSI) and Geometric Residual Gating (GRG).

Table 5: Ablation study of the proposed module on KITTI and nuScenes dataset.

Methods	Module		KITTI		nuScenes	
	LSI	GRG	Car	Ped	Car	Ped
Baseline	✗	✗	73.20 / 85.40	69.00 / 93.40	64.61 / 71.98	45.64 / 74.62
Variant A	✓	✗	74.10 / 86.20	68.70 / 93.10	65.22 / 72.43	46.27 / 75.10
Variant B	✗	✓	73.90 / 86.40	68.50 / 93.20	65.09 / 72.78	46.10 / 75.38
MAETrack	✓	✓	75.20 / 87.50	68.80 / 93.20	66.05 / 73.57	47.21 / 75.96

4.2.1. Overall Module Ablation.

We first investigate the contribution of the proposed LSI and GRG modules on both KITTI and nuScenes. As shown in Table 5, the proposed modules bring consistent improvements on the large-scale nuScenes benchmark. Specifically, on the Car category, LSI alone improves the baseline from 64.61/71.98 to 65.22/72.43, while GRG alone improves it to 65.09/72.78. Combining both modules further boosts the performance to 66.05/73.57. Similar trends can also be observed on the Pedestrian category, where MAETrack improves the baseline from 45.64/74.62 to 47.21/75.96. These results demonstrate that LSI and GRG are complementary: LSI reduces negative transfer from reconstruction-specific deep representations, while GRG reinforces structurally informative regions in the search feature.

On KITTI, MAETrack also significantly improves the Car category from 73.2/85.4 to 75.2/87.5, showing that the proposed transfer strategy is effective under limited-data settings. However, the gain on KITTI Pedestrian is less obvious, where the baseline remains slightly better. We attribute this to the small scale and category bias of KITTI, where the pedestrian subset contains highly sparse and non-rigid targets with limited

training diversity. Overall, the ablation results indicate that LSI and GRG jointly provide stable improvements, especially on the larger and more diverse nuScenes benchmark.

Table 6: Impact of different initialization strategies. Here, “Only Stage i ” means that only the i -th backbone stage is initialized from the BEV-MAE checkpoint, while all other stages are randomly initialized. “Stage 1+2+3” denotes cumulative initialization of the first three stages.

Initialization Strategy	KITTI		nuScenes	
	Car	Ped	Car	Ped
Baseline	73.20 / 85.40	69.00 / 93.40	64.61 / 71.98	45.64 / 74.62
Full Network Init.	73.80 / 86.10	68.30 / 92.80	65.09 / 72.58	46.22 / 75.24
Only Stage 1	74.40 / 86.50	68.60 / 93.00	65.39 / 72.90	46.13 / 75.11
Only Stage 2	73.60 / 85.90	68.40 / 92.90	64.69 / 71.56	46.42 / 75.23
Only Stage 3	72.90 / 85.10	68.20 / 92.70	63.65 / 71.23	46.22 / 74.45
Only Stage 4	72.40 / 84.60	68.00 / 92.60	63.14 / 69.60	46.01 / 75.00
Stage 1+2	75.00 / 87.20	68.70 / 93.10	66.10 / 73.01	46.58 / 75.42
Stage 1+2+3 (LSI)	75.20 / 87.50	68.80 / 93.20	66.05 / 73.57	47.21 / 75.96

4.2.2. Effect of Layer-Selective Initialization.

A core premise of MAETrack is the layer-wise transfer mismatch between reconstruction-based pre-training and downstream tracking. To examine this hypothesis more directly, we compare different initialization strategies in Table 6, including training from scratch, full-network initialization, single-stage initialization, and cumulative stage initialization. Here, “Only Stage i ” indicates that only the i -th backbone stage is initialized from the BEV-MAE checkpoint, while all other stages are randomly initialized. The results reveal clear layer-wise transfer differences. First, full-network initialization brings only limited gains over the baseline. For example, on nuScenes Car, it improves the baseline from 64.61/71.98 to 65.09/72.58, while on KITTI Car it improves from 73.20/85.40 to 73.80/86.10. This indicates that simply loading all pre-trained weights is not sufficient for effective tracking adaptation. Second, the single-stage initialization results show that different stages exhibit different transfer behaviors. Initializing early stages generally provides more stable improvements, while initializing deeper stages alone yields

limited or even degraded performance, especially on the Car category. For instance, on nuScenes Car, Only Stage 1 achieves 65.39/72.90, whereas Only Stage 3 and Only Stage 4 drop to 63.65/71.23 and 63.14/69.60, respectively. This observation supports our hypothesis that shallow layers preserve more transferable geometric cues, whereas deeper layers are more specialized to the reconstruction pretext task.

Furthermore, cumulative shallow-stage initialization provides a better overall transfer strategy. On KITTI Car, Stage 1+2+3 improves the baseline from 73.20/85.40 to 75.20/87.50. On nuScenes, Stage 1+2 obtains the highest Car Success, while Stage 1+2+3 achieves the best Car Precision and the best Pedestrian performance. Compared with full-network initialization, the default LSI setting improves nuScenes Car from 65.09/72.58 to 66.05/73.57 and Pedestrian from 46.22/75.24 to 47.21/75.96. These results indicate that the benefit of MAE pre-training does not simply come from using pre-trained weights, but from selectively inheriting layers that are more suitable for 3D SOT. Therefore, LSI provides a more appropriate adaptation strategy by preserving transferable shallow geometric priors while avoiding excessive dependence on reconstruction-specialized deep representations. It is worth noting that on KITTI Pedestrian, the baseline remains slightly better than the initialization-based variants. This suggests that the benefit of MAE transfer may vary across categories and datasets, especially under limited data and highly non-rigid pedestrian motion. Nevertheless, the consistent gains on KITTI Car and the larger-scale nuScenes benchmark demonstrate the effectiveness of layer-selective transfer for adapting MAE-pretrained representations to 3D tracking.

Table 7: Layer-wise representation diagnosis of BEV-MAE pre-trained features on nuScenes Car and Ped. CKA measures the similarity between BEV-MAE pre-trained features and tracking fine-tuned features at each stage. Foreground F1 and Boundary IoU are obtained by training a lightweight linear probe on frozen stage features to predict target occupancy and boundary regions in BEV space.

Stage	CKA Similarity $\uparrow$	Foreground F1 $\uparrow$	Boundary IoU $\uparrow$	Only-stage Init.	
Stage 1	0.84	72.8	44.6	65.39 / 72.90	46.13 / 75.11
Stage 2	0.71	70.9	42.7	64.69 / 71.56	46.42 / 75.23
Stage 3	0.52	66.3	38.9	63.65 / 71.23	46.22 / 74.45
Stage 4	0.36	62.5	35.1	63.14 / 69.60	46.01 / 75.00

4.2.3. Layer-wise Representation Diagnosis.

Although the initialization ablation in Table 6 provides performance-level evidence for the layer-wise transfer mismatch, it does not directly reveal what type of information is preserved at different stages. To further analyze the representation behavior of BEV-MAE pre-trained features, we conduct a layer-wise representation diagnosis on the nuScenes Car category.

Specifically, we evaluate two complementary aspects. First, we compute the linear CKA similarity between BEV-MAE pre-trained features and tracking fine-tuned features at each stage. Given two feature matrices X and Y extracted from the same stage of the pre-trained and fine-tuned backbones, respectively, the CKA similarity is computed as:

$$\text{CKA}(X, Y) = \frac{\|X^\top Y\|_F^2}{\|X^\top X\|_F \|Y^\top Y\|_F}. \quad (12)$$

A higher CKA value indicates that the representation is more consistently preserved after tracking fine-tuning. Second, we perform a lightweight geometric probing experiment. For each frozen stage feature, we train a 1×1 linear probe to predict two BEV-level geometric targets derived from the ground-truth 3D bounding box: a foreground occupancy mask and a boundary mask. Foreground F1 and Boundary IoU are used to evaluate whether the corresponding stage preserves tracking-relevant local geometric information. As shown in Table 7, shallow stages exhibit substantially higher CKA similarity than deeper stages. Stage 1 obtains the highest CKA similarity of 0.84, while Stage 4 decreases to 0.36. This indicates that shallow BEV-MAE representations remain more consistent with the downstream tracking representation, whereas deeper representations undergo larger shifts during fine-tuning. Such behavior suggests that deeper layers are more specialized to the reconstruction pretext task and require stronger adaptation to tracking-specific objectives.

The geometric probing results show a consistent trend. Stage 1 achieves the best Foreground F1 and Boundary IoU, indicating that shallow layers preserve more target-relevant occupancy and boundary structures. In contrast, deeper stages show weaker probing performance, suggesting that they contain less directly transferable local geometric information for precise target localization. These representation-level observations are consistent with the only-stage initialization results in Table 6, where Stage 1 provides

more stable transfer gains while Stage 3 and Stage 4 alone lead to degraded performance. Together, these results provide stronger evidence for our layer-wise transfer hypothesis: shallow MAE-pretrained layers mainly encode transferable geometric priors, while deeper layers are more reconstruction-specific. This further supports the design of LSI, which selectively inherits shallow stages while allowing deeper layers to adapt to the downstream 3D tracking objective.

Table 8: Comparison between GRG and generic attention/gating modules on nuScenes dataset. All modules are inserted into the search branch before template-search fusion under the same LSI initialization. A_c and A_s denote channel and spatial attention maps, respectively. R denotes the spatial gating map predicted by the gating module.

Method	Applied Branch	Modulation Form	Car		Pedestrian	
			Success	Precision	Success	Precision
LSI only	–	F_s	65.22	72.43	46.27	75.10
LSI + SE [31]	Search	$F_s \odot A_c$	65.36	72.66	46.38	75.22
LSI + CBAM [32]	Search	$F_s \odot A_c \odot A_s$	65.52	72.88	46.55	75.35
LSI + Spatial-only CBAM [32]	Search	$F_s \odot R$	65.68	73.05	46.73	75.47
LSI + Residual Spatial Gate [45]	Search	$F_s \odot (1 + R)$	65.86	73.31	46.98	75.71
LSI + GRG (Ours)	Search	$F_s \odot (1 + \alpha R)$	66.05	73.57	47.21	75.96

4.2.4. Comparison with Generic Attention and Gating Modules.

To clarify whether the improvement of GRG simply comes from adding a generic attention module, we compare GRG with representative attention and gating mechanisms, including SE, CBAM, spatial-only CBAM, and residual spatial gating. For a fair comparison, all modules are inserted into the same location, i.e., the search branch before template-search fusion, and are trained under the same LSI initialization strategy. As shown in Table 8, generic attention modules bring only limited improvements over LSI alone. SE mainly recalibrates channel responses and provides marginal gains, while CBAM and spatial-only CBAM further improve performance by introducing spatial modulation. However, directly applying spatial gating is still inferior to residual modulation, indicating that preserving the original MAE-pretrained geometric representation is important. The residual spatial gate improves stability, while the proposed GRG achieves the best performance by introducing a learnable residual strength α . These re-

sults suggest that GRG is not merely a generic attention block, but a geometry-preserving modulation mechanism tailored for MAE-pretrained 3D tracking features.

Table 9: Ablation on the placement of GRG on KITTI and nuScenes dataset.

GRG Placement	KITTI		nuScenes	
	Car	Ped	Car	Ped
Template Only	73.30 / 85.70	68.10 / 92.50	64.21 / 71.76	45.80 / 74.68
Both Branches	74.10 / 86.40	68.50 / 92.80	64.84 / 72.92	46.15 / 75.06
After Fusion	74.60 / 86.90	68.60 / 93.00	65.52 / 72.81	46.56 / 75.37
Search Only (Ours)	75.20 / 87.50	68.80 / 93.20	66.05 / 73.57	47.21 / 75.96

4.2.5. Design Choices in GRG.

GRG is designed to reinforce search-side geometric features before template-search fusion. To verify this design, we ablate the placement of GRG in Table 9, including applying it to the template branch only, both branches, after fusion, and the search branch only. The results show that applying GRG only to the search branch consistently achieves the best performance across both KITTI and nuScenes. For example, on KITTI Car, search-only GRG obtains 75.20/87.50, outperforming template-only, both-branch, and after-fusion variants. Similarly, on nuScenes Car, search-only GRG achieves 66.05/73.57, while applying GRG after fusion only obtains 65.52/72.81. This observation supports our design motivation. The template branch usually provides a relatively stable target reference, since it is cropped around the annotated or previously localized target. Applying gating to the template branch may disturb this reliable geometric reference. This explains why template-only modulation yields inferior results. Applying GRG to both branches partially improves over template-only modulation, but still risks perturbing the template representation.

In contrast, the search branch contains more background clutter, spatial ambiguity, and incomplete observations. Therefore, it benefits more from geometry-preserving residual modulation. Applying GRG after fusion is also sub-optimal, because the search features have already been fused with the template before structural reinforcement is performed. As a result, the fusion module has to match unenhanced search features,

making it harder to suppress background noise and emphasize target-related geometry. These results indicate that the effectiveness of GRG does not merely come from adding an extra gating operation. Instead, its benefit depends on applying residual geometric reinforcement to the search representation at the appropriate stage, before template-search fusion. This further supports our view that GRG is a search-oriented geometric modulation mechanism tailored for 3D SOT, rather than a generic spatial attention block.

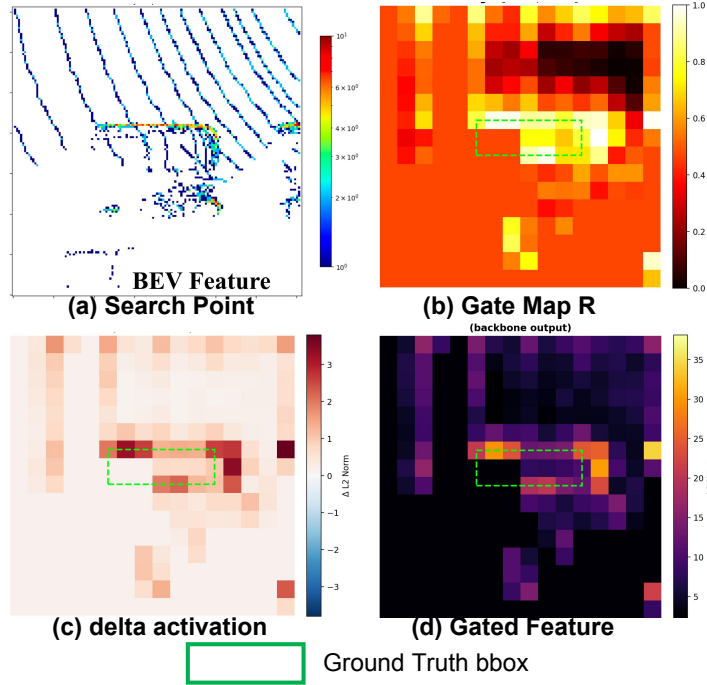


Figure 4: Visualization of GRG feature.

4.3. Visualization results

4.3.1. Feature Visualization of GRG

As shown in Figure 4, visualized results of the Gate Map (R) and the corresponding feature variation (Δ Activation) demonstrate the effectiveness of the GRG module. The modulated weight R ranges from 0.283 to 0.995, with peak values densely concentrated along the boundaries of the GT bbox. Consequently, the most significant feature amplifications (Δ Activation) are precisely aligned with the target’s vicinity. This

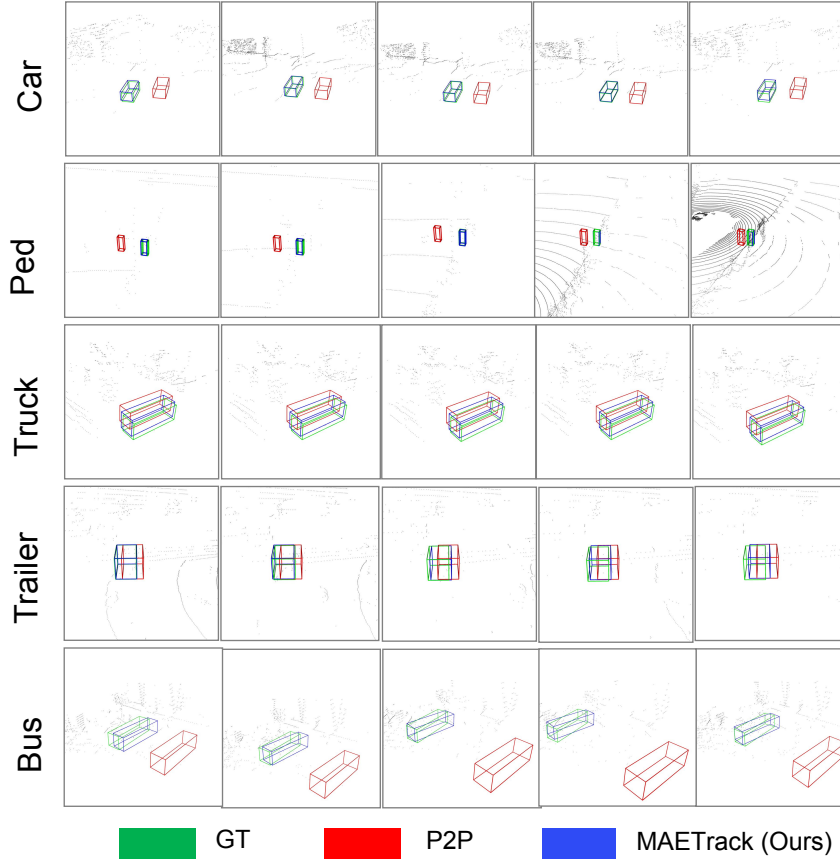


Figure 5: Tracking performance across varying classes on nuScenes [36] dataset.

indicates that GRG successfully achieves target-aware spatial modulation, explicitly reinforcing the boundary and local geometric structures of the object while mitigating uniform background distraction.

4.3.2. Tracking Visualization Across Diverse Object

Fig. 5 provides some visual comparison results over the SOTA baseline P2P [24] on nuScenes [36] dataset, across diverse trajectories. Whether in dense scenarios, or sparse scenarios, our MAETTrack is able to track the target, while P2P performs inaccurate box estimation. This highlights the superior tracking performance of our MAETTrack across

a range of conditions, from dense to sparse scenes. We also provide more visualization results in the appendix.

5. Conclusion

This paper presents **MAETrack**, a lightweight framework designed to bridge the gap between reconstructive 3D pre-training and downstream 3D SOT. We identify a layer-wise objective mismatch as a key obstacle: while shallow layers harbor transferable geometric cues, deeper layers specialize in reconstruction, which can impede tracking. To address this, MAETrack employs Layer-Selective Initialization (LSI) to inherit only the foundational geometric weights and Geometric Residual Gating (GRG) to reinforce structurally salient regions via spatial modulation. Benchmarks on KITTI and nuScenes demonstrate that MAETrack significantly outperforms vanilla fine-tuning, particularly for geometrically challenging categories, with negligible overhead. Our findings suggest that the true potential of 3D foundation models lies in their hierarchical geometric priors rather than final outputs. By reconciling these priors with temporal matching, we offer a principled approach to developing efficient, robust robotic perception systems.

Limitations and Future Work. Despite its effectiveness, MAETrack has several limitations: **(1)** MAETrack relies on a single pre-trained source; exploring multi-modal (camera-LiDAR) [34] or heterogeneous pre-training may yield richer priors. **(2)** GRG currently lacks explicit temporal modeling [46], which could be enhanced to handle fast motion or severe occlusion. **(3)** Our study focuses on CNN-based backbones; extending this paradigm to transformer-based [47, 29]. Future work will explore above issues and extending to multi-object tracking to further validate its generalization capabilities.

CrediT authorship contribution statement

Sifan Zhou: Investigation, Methodology, Software, Validation, Writing - original draft, Writing - review & editing, Project administration. **Qiwei Wang:** Methodology, Validation, Visualization, Writing - original draft, Writing - review & editing. **Linyue Tan:** Software, Formal analysis, Validation, Writing -original draft & editing. **Ziyu Liu:** Writing - original draft & editing. **Ziyu Zhao:** Writing - review & editing. **Ziyu**

Zhao: Formal analysis, Writing - review & editing. **Xiaobo Lu:** Methodology, Writing - review, Project administration, Funding acquisition.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (No. 62271143), the Frontier Technologies R&D Program of Jiangsu (No. BF2024060). On computing resources, this work was supported by the Big Data Computing Center of Southeast University.

References

- [1] B. Lu, Y. Sun, Z. Yang, R. Song, H. Jiang, Y. Liu, Hrnet: 3d object detection network for point cloud with hierarchical refinement, Pattern Recognition 149 (2024) 110254. [doi:10.1016/j.patcog.2024.110254](https://doi.org/10.1016/j.patcog.2024.110254).
- [2] Y. Shi, S. Zhou, W. Wang, X. Lu, Rethinking iterative stereo matching from a diffusion bridge model perspective, Pattern Recognition 167 (2025) 111737.
- [3] S. Zhou, L. Li, X. Zhang, B. Zhang, S. Bai, M. Sun, Z. Zhao, X. Lu, X. Chu, Lidar-ptq: Post-training quantization for point cloud 3d object detection, in: The Twelfth International Conference on Learning Representations (ICLR), 2024.
- [4] Y. Wang, others., [Target scanpath-guided 360-degree image enhancement](#), Proceedings of the AAAI Conference on Artificial Intelligence 39 (8) (2025) 8169–8177. [doi:10.1609/aaai.v39i8.32881](https://doi.org/10.1609/aaai.v39i8.32881).
URL <https://ojs.aaai.org/index.php/AAAI/article/view/32881>
- [5] Y. Cui, Z. Li, Z. Fang, Dynamic clustering transformer for lidar-based 3d object detection, Pattern Recognition 172 (2026) 112444. [doi:10.1016/j.patcog.2025.112444](https://doi.org/10.1016/j.patcog.2025.112444).
- [6] S. Zhao, S. Zhou, R. Blanchard, Y. Qiu, W. Wang, S. Scherer, Tartan imu: A light foundation model for inertial positioning in robotics, in: Proceedings of the Computer Vision and Pattern Recognition Conference, 2025, pp. 22520–22529.

- [7] Y. Pang, W. Wang, F. E. Tay, W. Liu, Y. Tian, L. Yuan, Masked autoencoders for point cloud self-supervised learning, in: European conference on computer vision, Springer, 2022, pp. 604–621.
- [8] C. Min, D. Zhao, L. Nie, W.-S. Nie, N. Qi, S. Honari, X. Li, et al., Voxel-mae: Masked autoencoders for pre-training large-scale point clouds, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 1723–1732.
- [9] M. Q. Khan, M. Shahzad, S. A. Khan, M. M. Fraz, X. X. Zhu, Beyond local patches: Preserving global-local interactions by enhancing self-attention via 3d point cloud tokenization, Pattern Recognition 155 (2024) 110712. [doi:10.1016/j.patcog.2024.110712](https://doi.org/10.1016/j.patcog.2024.110712).
- [10] S. Zhou, X. Zhang, X. Chu, B. Zhang, Z. Zhao, X. Lu, Fastpillars: A deployment-friendly pillar-based 3d detector, IEEE Transactions on Circuits and Systems for Video Technology (2025). [doi:10.1109/TCSVT.2025.3633725](https://doi.org/10.1109/TCSVT.2025.3633725).
- [11] S. Zhou, Z. Yuan, D. Yang, X. Hu, J. Qian, Z. Zhao, Pillarhist: A quantization-aware pillar feature encoder based on height-aware histogram, in: Proceedings of the Computer Vision and Pattern Recognition Conference, 2025, pp. 27336–27345.
- [12] C. Zheng, X. Yan, H. Zhang, B. Wang, S. Cheng, S. Cui, Z. Li, An effective motion-centric paradigm for 3d single object tracking in point clouds, IEEE Transactions on Pattern Analysis and Machine Intelligence (2023).
- [13] L. Zhao, Y. Hu, X. Yang, Y. Wang, Z. Dou, Y. Zhang, Voxel pillar multi-frame cross attention network for sparse point cloud robust single object tracking, Pattern Recognition 167 (2025) 111771. [doi:10.1016/j.patcog.2025.111771](https://doi.org/10.1016/j.patcog.2025.111771).
- [14] Y. Cui, Z. Fang, J. Shan, Z. Gu, S. Zhou, 3d object tracking with transformer, British Machine Vision Conference (2021) 1445–1458.
- [15] Y. Cui, Z. Fang, S. Zhou, Point siamese network for person tracking using 3d point clouds, Sensors 20 (1) (2019) 143.

- [16] Z. Hu, S. Zhou, J. Nie, Z. Zhao, W. Li, C.-j. Liang, Tftrack: A template-free framework for efficient 3d point cloud tracking, arXiv preprint arXiv:2609.07738 (2026).
- [17] S. Giancola, J. Zarzar, B. Ghanem, S. Giancola, J. Zarzar, Leveraging shape completion for 3d siamese tracking, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2019, pp. 1359–1368.
- [18] Z. Fang, S. Zhou, Y. Cui, S. Scherer, 3d-siamrpn: An end-to-end learning method for real-time 3d single object tracking using raw point cloud, IEEE Sensors Journal 21 (4) (2020) 4995–5011.
- [19] H. Qi, C. Feng, Z. Cao, F. Zhao, Y. Xiao, P2b: Point-to-box network for 3d object tracking in point clouds, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2020, pp. 6329–6338.
- [20] C. Zheng, X. Yan, J. Gao, W. Zhao, W. Zhang, Z. Li, S. Cui, Box-aware feature enhancement for single object tracking on point clouds, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 13199–13208.
- [21] J. Shan, S. Zhou, Z. Fang, Y. Cui, Ptt: Point-track-transformer module for 3d single object tracking in point clouds, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2021, pp. 1310–1316.
- [22] T.-X. Xu, Y.-C. Guo, Y.-K. Lai, S.-H. Zhang, Mbptrack: Improving 3d point cloud tracking with memory networks and box priors, arXiv preprint arXiv:2303.05071 (2023).
- [23] C. Zheng, X. Yan, H. Zhang, B. Wang, S. Cheng, S. Cui, Z. Li, Beyond 3d siamese tracking: A motion-centric paradigm for 3d single object tracking in point clouds, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 8111–8120.
- [24] J. Nie, F. Xie, S. Zhou, X. Zhou, D.-K. Chae, Z. He, P2p: Part-to-part motion cues guide a strong tracking framework for lidar point clouds, International Journal of Computer Vision (2025) 1–17.

- [25] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, R. Girshick, Masked autoencoders are scalable vision learners, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 16000–16009.
- [26] X. Tian, H. Ran, Y. Wang, H. Zhao, Geomae: Masked geometric target prediction for self-supervised point cloud pre-training, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 13570–13580. doi:[10.1109/CVPR52729.2023.01304](https://doi.org/10.1109/CVPR52729.2023.01304).
- [27] Z. Lin, Y. Wang, S. Qi, N. Dong, M.-H. Yang, Bev-mae: Bird’s eye view masked autoencoders for point cloud pre-training in autonomous driving scenarios, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 3531–3539. doi:[10.1609/aaai.v38i4.28141](https://doi.org/10.1609/aaai.v38i4.28141).
- [28] J. Shan, S. Zhou, Y. Cui, Z. Fang, Real-time 3d single object tracking with transformer, IEEE Transactions on Multimedia 25 (2022) 2339–2353.
- [29] S. Zhou, J. Nie, Z. Zhao, Y. Cao, X. Lu, Focustrack: One-stage focus-and-suppress framework for 3d point cloud object tracking, in: Proceedings of the 33rd ACM International Conference on Multimedia, Association for Computing Machinery, New York, NY, USA, 2025, p. 7366–7375. doi:[10.1145/3746027.3754781](https://doi.org/10.1145/3746027.3754781).
- [30] S. Zhou, W. Xu, J. Xiong, Z. Zhao, Z. Yuan, Pillartrack: Boosting pillar representation for transformer-based 3d single object tracking on point clouds, Knowledge-Based Systems 352 (2026) 116973. doi:<https://doi.org/10.1016/j.knosys.2026.116973>.
- [31] J. Hu, L. Shen, G. Sun, Squeeze-and-excitation networks, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 7132–7141.
- [32] S. Woo, J. Park, J.-Y. Lee, I. S. Kweon, Cbam: Convolutional block attention module, in: Proceedings of the European conference on computer vision (ECCV), 2018, pp. 3–19.

- [33] L. Hui, L. Wang, M. Cheng, J. Xie, J. Yang, 3d siamese voxel-to-bev tracker for sparse point clouds, *Advances in Neural Information Processing Systems* 34 (2021) 28714–28727.
- [34] Z. Hu, S. Zhou, Z. Yuan, D. Yang, S. Zhao, C.-j. Liang, Mvctrack: Boosting 3d point cloud tracking via multimodal-guided virtual cues, in: *2025 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2025, pp. 3745–3751.
- [35] A. Geiger, P. Lenz, R. Urtasun, Are we ready for autonomous driving? the kitti vision benchmark suite, in: *2012 IEEE conference on computer vision and pattern recognition*, IEEE, 2012, pp. 3354–3361.
- [36] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, O. Beijbom, nuscenes: A multimodal dataset for autonomous driving, in: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 11621–11631.
- [37] Y. Wu, J. Lim, M.-H. Yang, Online object tracking: A benchmark, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2013, pp. 2411–2418.
- [38] M. Kristan, J. Matas, A. Leonardis, T. Vojř, R. Pflugfelder, G. Fernandez, G. Nebehay, F. Porikli, L. Čehovin, A novel performance evaluation methodology for single-target trackers, *IEEE transactions on pattern analysis and machine intelligence* 38 (11) (2016) 2137–2155.
- [39] J. Shan, S. Zhou, Z. Fang, Y. Cui, Ptt: Point-track-transformer module for 3d single object tracking in point clouds, in: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2021, pp. 1310–1316.
- [40] C. Zhou, Z. Luo, Y. Luo, T. Liu, L. Pan, Z. Cai, H. Zhao, S. Lu, Pttr: Relational 3d point cloud object tracking with transformer, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 8531–8540.

- [41] J. Nie, Z. He, Y. Yang, M. Gao, J. Zhang, Glt-t: Global-local transformer voting for 3d single object tracking in point clouds, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2023, pp. 1957–1965.
- [42] T. Ma, M. Wang, J. Xiao, H. Wu, Y. Liu, Synchronize feature extracting and matching: A single branch framework for 3d object tracking, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 9953–9963.
- [43] Z. Luo, C. Zhou, L. Pan, G. Zhang, T. Liu, Y. Luo, H. Zhao, Z. Liu, S. Lu, Exploring point-bev fusion for 3d point cloud object tracking with transformer, IEEE Transactions on Pattern Analysis and Machine Intelligence (2024).
- [44] Y. Lu, J. Nie, Z. He, H. Gu, X. Lv, Voxeltrack: Exploring multi-level voxel representation for 3d point cloud object tracking, in: Proceedings of the 32nd ACM International Conference on Multimedia, 2024, pp. 6345–6354.
- [45] F. Wang, M. Jiang, C. Qian, S. Yang, C. Li, H. Zhang, X. Wang, X. Tang, Residual attention network for image classification, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2017, pp. 3156–3164.
- [46] B. Fan, S. Zhou, J. Li, S. Zhao, M. Cao, Q. Wang, Beyond frame-wise tracking: A trajectory-based paradigm for efficient point cloud tracking, IEEE Robotics and Automation Letters (2026).
- [47] S. Zhou, Y. Cao, J. Nie, Y. Fu, Z. Zhao, X. Lu, S. Wang, Comptrack: Information bottleneck-guided low-rank dynamic token compression for point cloud tracking, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, 2026, pp. 13773–13781. [doi:10.1609/aaai.v40i16.38385](https://doi.org/10.1609/aaai.v40i16.38385).